



---

# Internship Report (Extra)

Detection and Semantic Segmentation on Meta  
Quest 3

---

NGUYEN Hoang Nhat

Internship Period: April 20, 2026 – June 19, 2026

Laboratoire Mathématiques, Image et Applications  
La Rochelle Université

Submitted on:  
29/5/2026

Supervisor:  
Péteri Renaud  
University tutor:  
Mascarilla Laurent

# Contents

## 1 Code Explanation

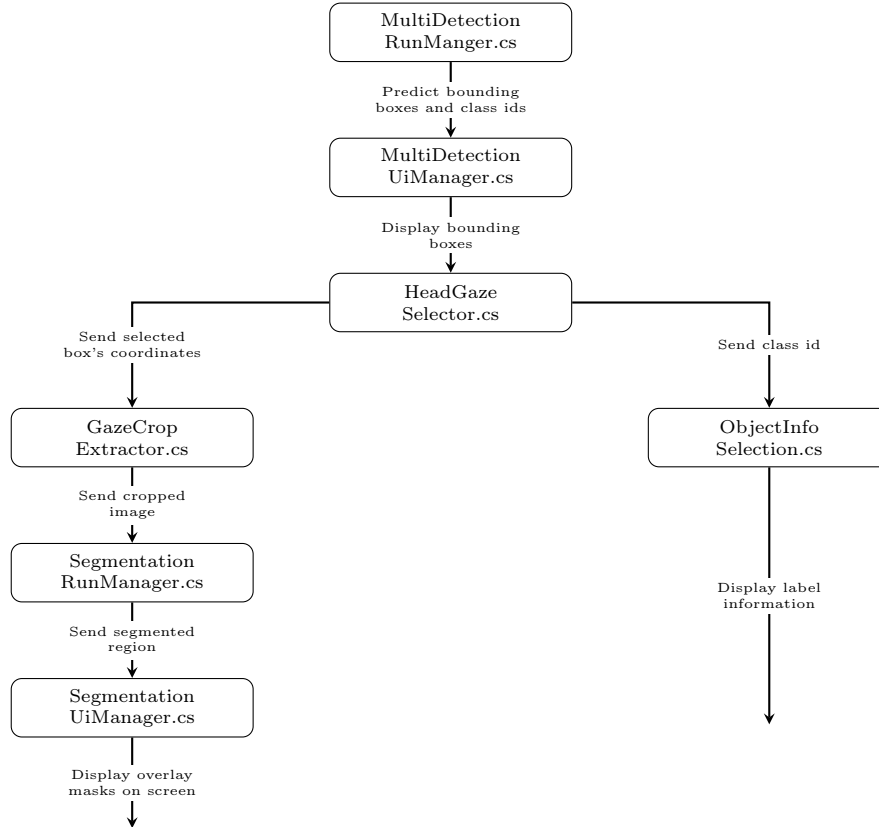
- 1.1 Execution Flow . . . . .
- 1.2 Detection Stage . . . . .
  - 1.2.1 `MultiDetectionRunManager.cs` - Inference Loop . . . . .
  - 1.2.2 `MultiDetectionUiManager.cs` - Bounding Box Rendering . . . . .
- 1.3 Gaze Selection . . . . .
  - 1.3.1 `HeadGazeSelector.cs` - Gaze Ray Casting . . . . .
- 1.4 Right Branch - Object Information Display . . . . .
  - 1.4.1 `ObjectInfoDatabase.cs` and `ObjectInfoEntry.cs` - JSON Metadata . . . . .
  - 1.4.2 `ObjectInfoSelection.cs` - Info Panel . . . . .
- 1.5 Left Branch - Segmentation Pipeline . . . . .
  - 1.5.1 `GazeCropExtractor.cs` - ROI Cropping and Letterboxing . . . . .
  - 1.5.2 `SegmentationRunManager.cs` - Segmentation Inference . . . . .
  - 1.5.3 `SegmentationUiManager.cs` - Mask Overlay Rendering . . . . .

## 2 Fixes

- 2.1 Finishing Model finetuning . . . . .
  - 2.1.1 Detection . . . . .
  - 2.1.2 Semantic Segmentation . . . . .

# 1 Code Explanation

## 1.1 Execution Flow



The pipeline is split into two parallel stages. The detection stage runs continuously on every other camera frame, while the segmentation stage is triggered on demand only when the user selects a bounding box with their head gaze. The following subsections describe each script in pipeline order.

## 1.2 Detection Stage

### 1.2.1 MultiDetectionRunManager.cs - Inference Loop

This script is the entry point of the real-time detection pipeline. Every frame it retrieves the live passthrough camera texture from the `PassthroughCameraAccess`, copies it into a pre-allocated `Tensor<float>` of shape `(1, 3, 1280, 1280)`, and schedules it through a Unity `Sentis Worker` backed by the Vulkan compute backend (`GPUCompute`).

The ONNX model (YOLOv8n) is converted to the `.sentis` format in the Editor by `DetectionModelEditorConverter.cs`, which appends a centers-to-

corners matrix multiplication layer so that bounding-box coordinates arrive in  $(x_1, y_1, x_2, y_2)$  corner format rather than the native  $(c_x, c_y, w, h)$  format. This avoids the conversion cost at runtime on every frame.

The model produces three output tensors - corners, class IDs, and confidence scores - whose GPU-to-CPU readbacks are issued simultaneously and awaited in parallel inside a coroutine, minimising GPU pipeline stalls. Once the data arrives on the CPU, a standard greedy Non-Maximum Suppression (NMS) pass is applied using pre-allocated working lists (no per-frame heap allocations), and the surviving detections are forwarded to `MultiDetectionUiManager`.

### 1.2.2 `MultiDetectionUiManager.cs` - Bounding Box Rendering

This script translates the 2D pixel-space detections into world-space bounding boxes anchored to real surfaces. For each detection, the normalised viewport center is converted to a 3D ray via `PassthroughCameraAccess.ViewportPointToRay`, which internally uses the camera's calibrated intrinsics. The ray is then intersected with the environment mesh by `EnvironmentRayCastSampleManager` to obtain a world-space hit point.

To determine the physical size of the box, the four corner rays of the detection rectangle are intersected with a plane perpendicular to the camera direction at the depth of the hit point. The resulting world-space corner offsets are projected into the camera's local coordinate frame to extract a 2D width and height. A `RectTransform` prefab is then placed at the hit point, oriented to face the camera, and sized to these world-space dimensions.

A pooling system avoids instantiating new `GameObjects` on every frame: existing boxes are reused when a new detection overlaps them ( $\text{IoU} > 0$ ) and belongs to the same class, or removed when a different class occupies the same region ( $\text{IoU} > 0.1$ ). Boxes that have not been updated for 750 ms are automatically returned to the pool.

Each `BoundingBoxData` record also stores the detection's normalised bounding-box coordinates (the `NormalizedBBBox` field), which are passed downstream to the crop extractor.

## 1.3 Gaze Selection

### 1.3.1 `HeadGazeSelector.cs` - Gaze Ray Casting

This script implements a hands-free selection mechanism. Each frame it constructs a ray from the `CenterEyeAnchor` transform (representing the user's head direction) and tests it against every active bounding box managed by `MultiDetectionUiManager`.

For each box, the perpendicular distance from the gaze ray to the box's world-space center is compared against a hit radius defined as half the diagonal of the box's world-space size. Among all boxes that pass this test, the one whose center subtends the smallest angle with the gaze direction is selected. A color change on the box's `Image` component provides visual feedback, and

an `OnSelectionChanged` event is fired whenever the selection changes. The two downstream consumers - `GazeCropExtractor` and `ObjectInfoSelection` - both subscribe to this event.

## 1.4 Right Branch - Object Information Display

### 1.4.1 `ObjectInfoDatabase.cs` and `ObjectInfoEntry.cs` - JSON Metadata

`ObjectInfoDatabase` parses a JSON asset at startup. The root object is expected to contain a "classes" array; any other top-level keys (e.g. "metadata") are ignored. Each element is deserialised into an `ObjectInfoEntry` instance using `Newtonsoft.Json` and stored in a dictionary keyed by integer class ID.

`ObjectInfoEntry` is the data model. It captures per-class metadata used for display: canonical name, aliases, material composition, 3D shape profile (geometry and symmetry), typical physical dimensions in centimetres, segmentation difficulty and recommended approach, occlusion risk, and pose estimation notes.

### 1.4.2 `ObjectInfoSelection.cs` - Info Panel

When `HeadGazeSelector` fires `OnSelectionChanged`, this script looks up the selected class ID in `ObjectInfoDatabase` and formats a rich-text string (using Unity's legacy `Text` component with bold and italic tags). The info panel - a world-space `Canvas` child - is then activated, sized to fixed world-space dimensions, and positioned flush to the right edge of the selected bounding box with a configurable gap. Each frame the panel follows the box using a `Vector3.Lerp` and billboards to face the main camera.

## 1.5 Left Branch - Segmentation Pipeline

### 1.5.1 `GazeCropExtractor.cs` - ROI Cropping and Letterboxing

This script runs at a configurable interval (default 100 ms) rather than every frame to avoid saturating the GPU with segmentation inference requests. When the gaze selection is non-null, it crops the padded bounding-box region from the live camera texture using two sequential GPU `Graphics.Blit` calls.

The first blit extracts the raw crop at native pixel resolution. The second applies a letterbox resize: a uniform scale factor  $s = \min(1, T/w, T/h)$  is computed where  $T = 640$  is the model input size and  $w, h$  are the crop dimensions. This ensures larger crops are scaled down to fit while smaller crops are padded without upscaling. The output is always a  $640 \times 640$  `RenderTexture` with black padding bars on the shorter axis.

Both the viewport coordinates of the original padded bbox in camera space and the UV sub-rect of the real-image content inside the  $640 \times 640$  texture are stored in a `CropInfo` struct and forwarded to `SegmentationRunManager` via the `OnCropReady` event. The UV sub-rect is essential for the overlay placement

step, allowing the downstream UI manager to display only the content region of the mask texture.

### 1.5.2 SegmentationRunManager.cs - Segmentation Inference

This script supports two operating modes selectable via a single Inspector toggle.

**Semantic mode (YOLO26n-sem).** The model outputs a single tensor of shape  $(1, C, H, W)$  containing per-pixel class logits. An argmax over the class dimension  $C$  produces a flat integer array `classMap` of size  $H \times W$ . A validity check ensures the target class ID (the one gaze-selected by the user) is present in the map before the result is fired; frames where the object is absent are silently dropped.

**(Optional) Instance mode (YOLO26n-seg).** The model outputs two tensors: detections of shape  $(1, 4 + C + 32, 8400)$  and prototype masks of shape  $(1, 32, H_p, W_p)$ . Only anchors whose score for the target class exceeds a configurable threshold are kept. These are sorted by score and deduplicated with NMS. For the surviving best detection, its 32 mask coefficients are combined with the prototype masks via a dot product followed by a sigmoid, producing a continuous mask at prototype resolution. Pixels above a configurable threshold (default 0.5) are labelled with the target class ID; all others are set to background. The mask is then cropped to the detection bounding box projected onto prototype space to suppress leakage outside the object extent.

Both modes produce a `SegResult` struct with the same fields - `classMap`, map dimensions, target class ID, class label, the crop viewport rect, and the letterbox UV rect - so the downstream UI manager is mode-agnostic.

A debug placeholder mode bypasses inference entirely and fires a placeholder `SegResult` (with a null `classMap`), allowing the UI placement logic to be verified independently of a working model.

### 1.5.3 SegmentationUiManager.cs - Mask Overlay Rendering

Upon receiving a `SegResult`, this script offloads pixel colouring to a background thread-pool task (`Task.Run`) to avoid blocking the Unity main thread. The task iterates over the `classMap` and writes a semi-transparent color into a `Color32` array: pixels whose class ID matches the target receive an HSV-derived tint unique to that class ID; all others are written as fully transparent. A vertical flip is applied during iteration because model output row 0 is the top of the image while `Texture2D` row 0 is the bottom.

When the task completes the main thread uploads the pixel array to a reused `Texture2D` and calls `PlaceOverlay`. This method replicates the same world-space projection logic used for bounding boxes: the crop viewport corners are cast as rays, intersected with a plane at the environment-raycast depth, and the resulting world-space size is assigned to the overlay `RectTransform`'s `sizeDelta`. Crucially, `RawImage.uvRect` is set to `ModelInputUVRect` so that

only the letterbox-content region of the  $640 \times 640$  texture is displayed, making the mask pixel-accurately aligned with the underlying bounding box.

A serial number is incremented each time a new result arrives; stale completions (from a previous result whose task finished after a newer one began) are discarded before the texture upload, preventing flickering from out-of-order deliveries.

## 2 Fixes

This section describes the work done after the internship defense and towards the end of my internship.

### 2.1 Finishing Model finetuning

The dataset's images were changed to match the size of the frame input: 1280x1280 for detection and 640x640 for segmentation. The script for resizing can be found in **My Github**

#### 2.1.1 Detection

Every hyperparameter and data augmentation configuration were kept the same, except for the 'imgsize' which is now set to 640x640.

**Results.** The fine-tuned model was evaluated during training on the validation set at IoU threshold 0.5 and 0.5-0.95, which is shown in Figure ???. The resulting model was then evaluated on the held-out test set, and per-class metrics are reported in Table ??.

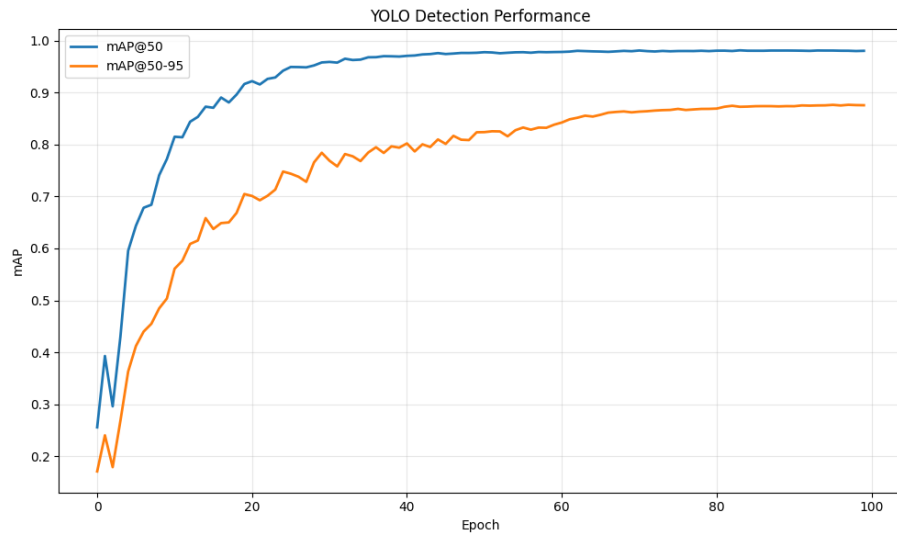


Figure 1: Mean Average Precision at threshold 0.5 and 0.5-0.95 on the validation set during training over 100 epochs

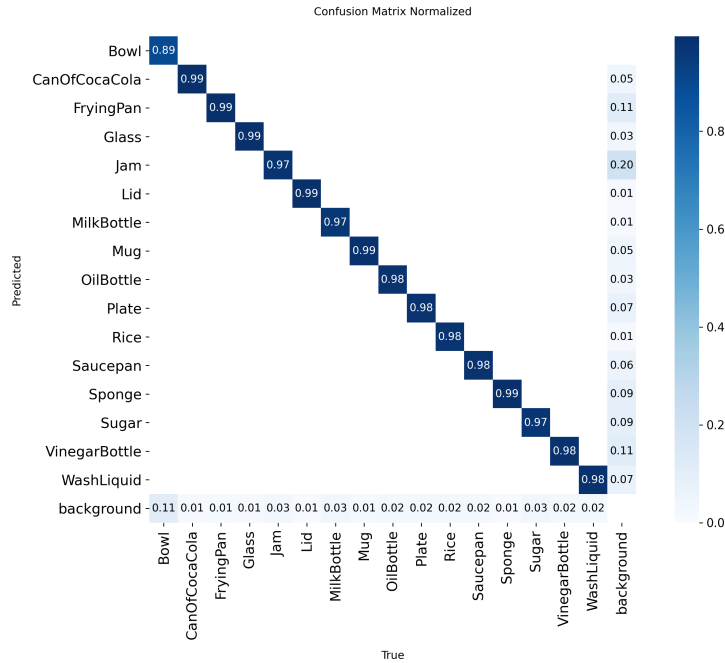


Figure 2: Normalized Confusion matrix on the validation set



### 2.1.2 Semantic Segmentation

Every hyperparameter and data augmentation configuration were kept the same, except for the ‘imgsize’ which is now set to 1280x1280. Note that this matches the resolution of the newest **HorizonOS v83**.

**Results.** The fine-tuned model was evaluated on the held-out test set. Training and validation curves are shown in Figure 3. The results are excellent, reaching a mIoU of **0.98**.

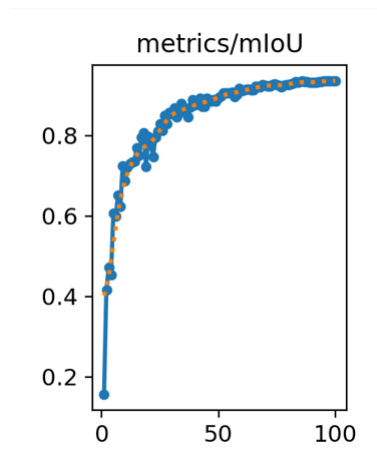


Figure 3: Mean Intersection over Union (mIoU) on the validation set during training over 100 epochs

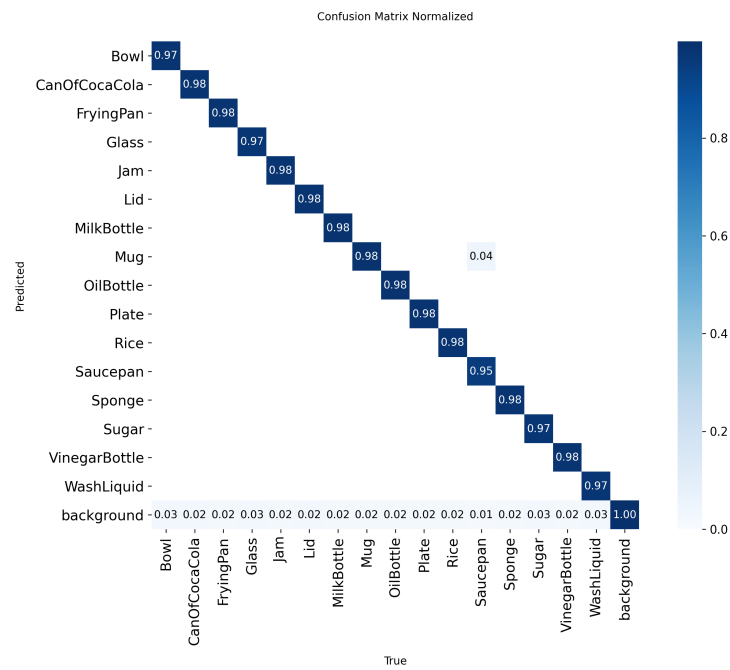


Figure 4: Normalized Confusion matrix on the validation set